



SOLVING THE LORENZ ODE SYSTEM USING OPTIMAL ANN ARCHITECTURES

Ikramul Hasan Moral • Md. Abu Bakar • Samiur Rahman Omlan • Fariha Islam •
Md. Touhidul Islam

Department of Computer Science and Engineering, United International University
Supervisor: Dr. Muhammad Nomani Kabir, Professor, Dept. of CSE • **Course Teacher:** Dr. Md. Motaharul Islam, Professor, Dept. of CSE

Introduction & Central Question

Coupled nonlinear ODEs such as the **Lorenz-1960** system — an early three-equation meteorological model — rarely admit closed-form solutions. Classical Runge–Kutta solvers are accurate but **mesh-dependent** and re-integrate step-by-step for every query. A trained network can instead learn the continuous map $t \mapsto (x, y, z)$ in one shot, yet most neural-ODE work leans on physics-informed losses or operator learning.

Central question

Can a **plain feedforward ANN** — trained only on numerical data, with *no physics in the loss* — approximate the Lorenz-1960 solution, and *which architecture does it best?*

Objectives. (i) build a trusted RK4+SciPy reference and cross-validate it; (ii) vary **depth** $\times$ **width** $\times$ **activation** systematically; (iii) score accuracy vs. cost (MSE, MAE, max error, train/infer time); (iv) benchmark the best plain ANN against PINN / DeepONet results.

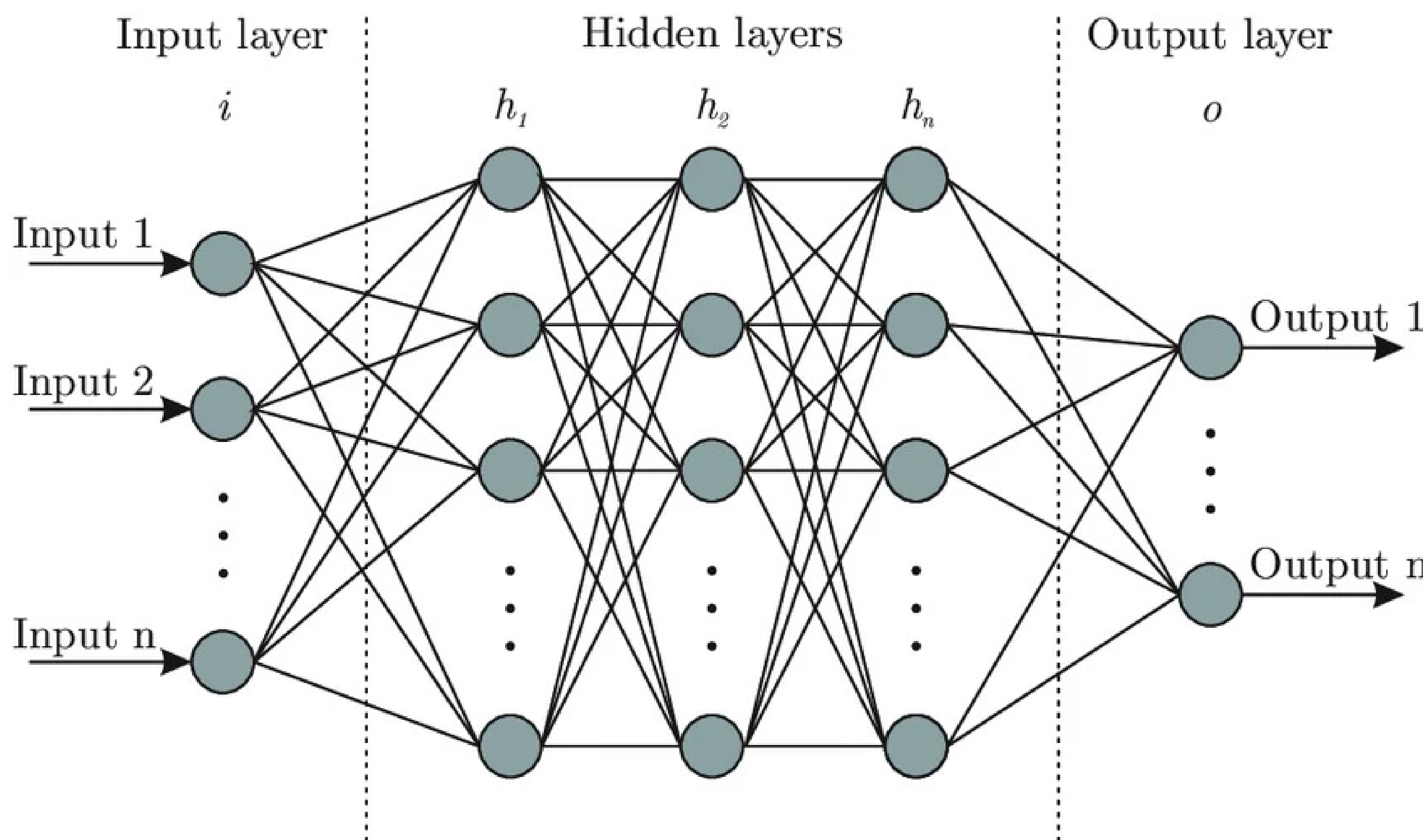
Background: System & Network

Three coupled nonlinear ODEs; with $k = 2$, $l = 1$:

$$\begin{aligned}\dot{x} &= kl \left(\frac{1}{k^2 + l^2} - \frac{1}{k^2} \right) yz \\ \dot{y} &= kl \left(\frac{1}{l^2} - \frac{1}{k^2 + l^2} \right) xz \\ \dot{z} &= \frac{kl^2}{2} \left(\frac{1}{k^2} - \frac{1}{l^2} \right) xy\end{aligned}$$

Reduced form: $\dot{x} = -0.1 yz$, $\dot{y} = 1.6 xz$, $\dot{z} = -0.75 xy$.

Initial state $x_0=0.5$, $y_0=0.75$, $z_0=1$, on $t \in [0, 1]$.



Plain feedforward ANN: learns $t \mapsto (x, y, z)$ by backpropagation. PINNs appear only as a literature benchmark, not as part of this model.

Methodology — Two-Phase Study (69 Controlled Runs)

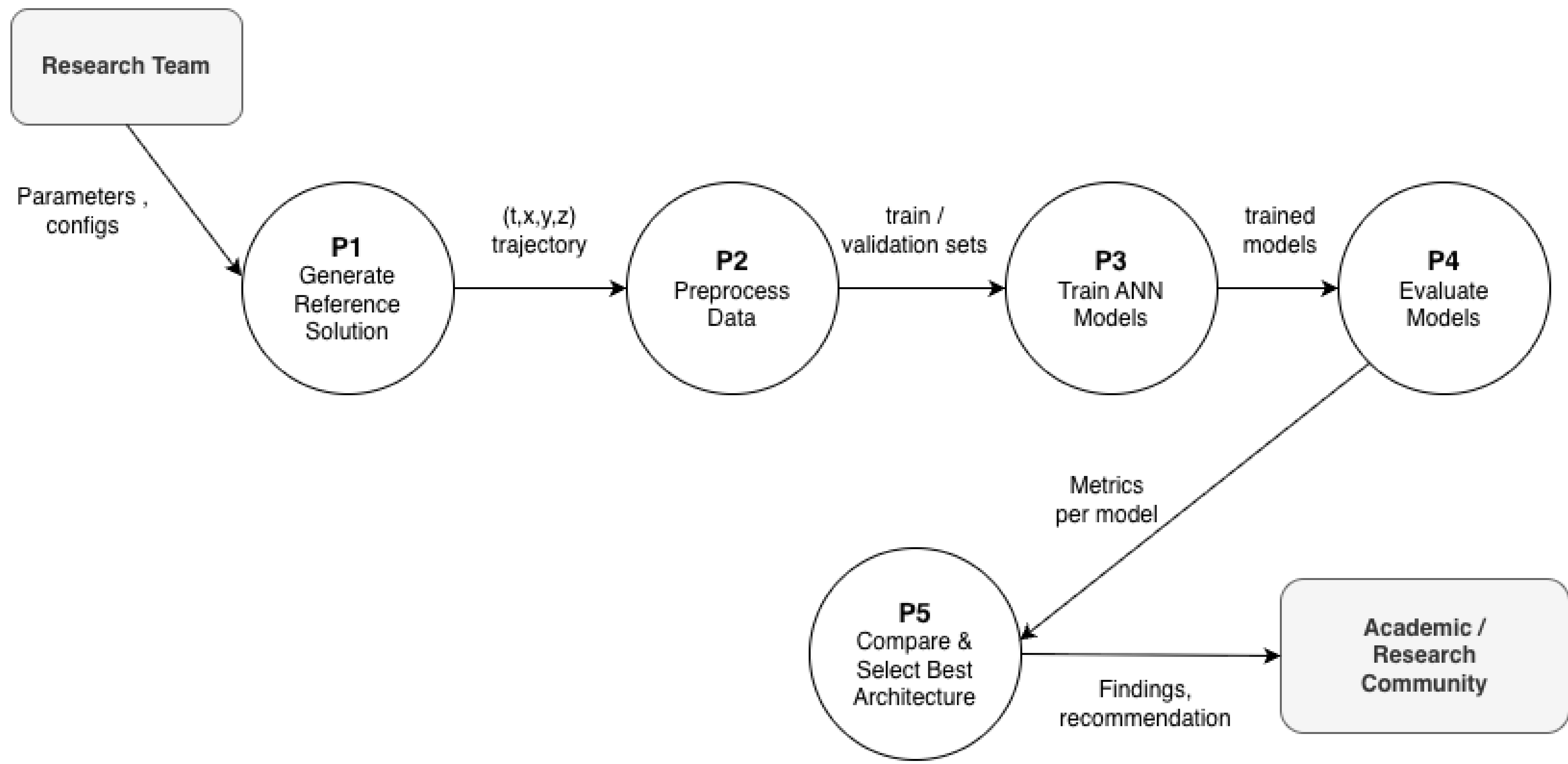
Phase 1 — Architecture search

depth $\{1, 2, 3, 4\} \times$ width $\{20, 50, 100\} \times$ activation $\{\tanh, \text{ReLU}, \text{sigmoid}, \text{GELU}, \text{Swish}\}$
= **60 runs** (Adam held fixed)

Phase 2 — Optimizers

3 best Phase-1 architectures $\times$ $\{\text{Adam}, \text{L-BFGS}, \text{SGD} + \text{momentum}\}$
= **9 runs**

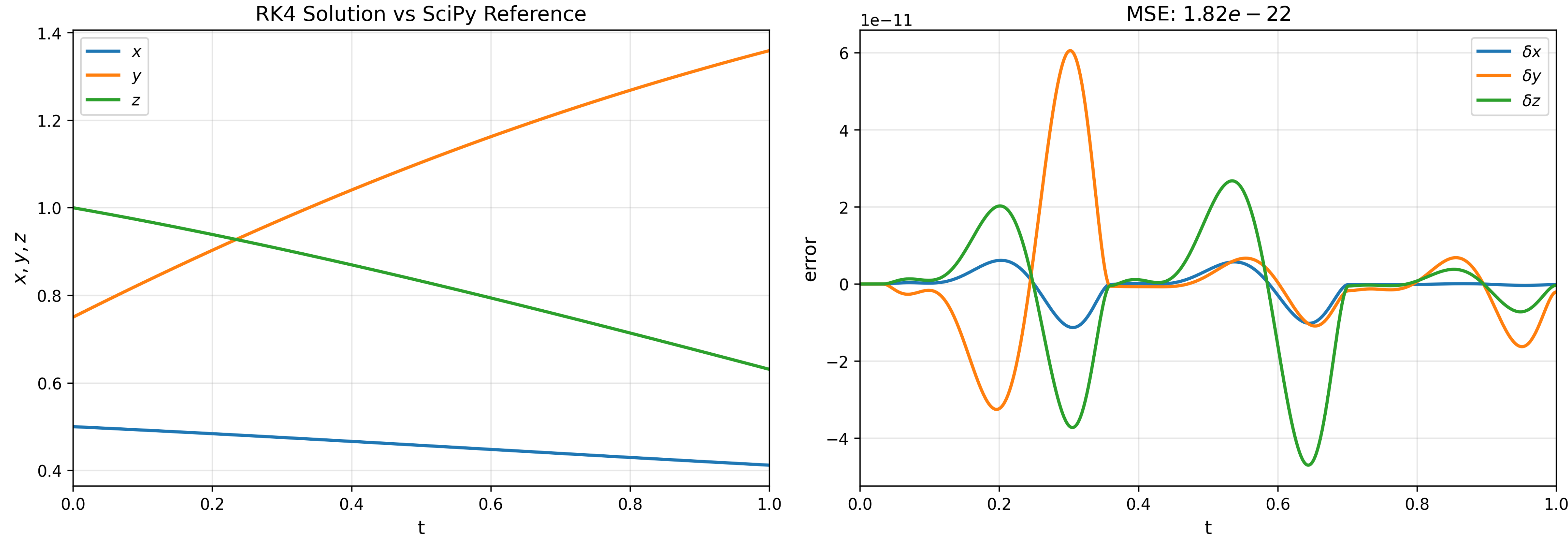
69 controlled experiments — the split cleanly separates *architecture* effects from *optimizer* effects.



Data pipeline: generate $\rightarrow$ preprocess $\rightarrow$ train $\rightarrow$ evaluate $\rightarrow$ select.

Numerical Baseline & Validation

The reference trajectory is produced by two independent solvers and cross-checked *before any ANN sees it*: a custom **RK4** integrator ($h=10^{-3}$, 1001 points) and **SciPy DOP853** ($\text{rtol}=10^{-10}$), split 80/20 for train/validation (seed 42).



The two solvers agree to **MSE** $\approx 1.8 \times 10^{-22}$, far below the 10^{-10} target — a defensible **ground truth** for all FYDP-II ANN training.

Status (FYDP-I) & Future Work

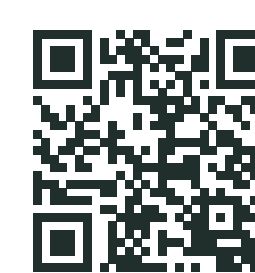
Delivered this stage: research question framed; 17-paper review with an explicit gap; two-phase design (69 runs) locked; dual-solver baseline verified. **No accuracy claims are made** until FYDP-II experiments produce evidence.

- Build the ANN training pipeline; run Phase 1 (60), then Phase 2 (9).
- Repeat seeds for stability; test denser grids and new initial conditions.
- Compare directly with PINN / operator-learning baselines.

Key References & Contact

[1] Lorenz (1960), *Tellus* 12(3). [2] Matthews & Bihlo (2024), PinnDE. [3] Raissi et al. (2019), *J. Comput. Phys.* 378.
[4] Lagaris et al. (1998), *IEEE TNN* 9(5). [5] Lau & Werth (2020), ODEIN.

Team_ParaDox • Project repository • github.com/ihmorol/fydp_workspace



Scan for code & data